

Split Complex Lorentzian Dynamics

Part XXI: The Quaternionic Double-Cover Identification in an Interrupted Formal Run
Source-level closure of the core and central-product layers, the exact timeout boundary, and why
Task XXII must verify rather than extend

Editor: Jan Seeck

AI Assistant: OpenAI ChatGPT (GPT-5.6 Sol)

Lean Agent: Aristotle (Harmonic)

4 September 2026

Abstract

Task 21 was designed as a narrow identification task after the broad equivalence audit of Task 20. It did not finish. The Aristotle run reached its time limit before a final endpoint module, audit, focused build report, whole-project build report, or axiom report was produced. The exported archive nevertheless contains a substantial new source layer: thirteen Task-21 Lean modules comprising 2655 lines. Twelve of those modules contain no occurrence of `sorry`; all twenty-four occurrences of `sorry`, on twenty-two source lines, are confined to one file, `NormalizedCentralCarrier.lean`. The interruption therefore occurred late and at a sharply identifiable mathematical boundary rather than near the beginning of the task.

The source-level results before that boundary are substantial. The Task-20 identification of the intrinsic core with unit quaternions is repackaged as an explicit multiplicative equivalence and homeomorphism. A standard 2×2 complex-matrix model is then constructed and used to define an explicit group equivalence with $SU(2)$. Independently, the intrinsic visible group is represented on the reconstructed three-dimensional spatial carrier, its matrices are proved orthogonal with determinant $+1$, and the source contains an explicit group equivalence

$$G_{\text{vis}} \simeq SO(3).$$

The intrinsic core projection is compared pointwise with quaternionic conjugation on the imaginary quaternion space, yielding a commuting projection statement and a direct source-level proof that the standard quaternionic rotation kernel is exactly $\{\pm 1\}$. The section branch is also repackaged: a set-theoretic section exists, whereas a continuous section is ruled out.

The enlarged central carrier is likewise advanced far beyond the Task-20 comparison stage. The exact central plane is mapped explicitly to $\mathbb{C}$ and its invertible subgroup is packaged as a multiplicative equivalence with $\mathbb{C}^\times$. The intersection of this centre with the quaternionic core is proved source-level to be exactly $\{\pm 1\}$. The multiplication map from centre times core onto the full internal carrier is defined, proved surjective in the source, and assigned the diagonal two-element kernel. The resulting quotient equivalence

$$\mathcal{L}_Z \simeq (\mathbb{C}^\times \times \mathbb{H}_1) / \Delta \mathbb{Z}_2$$

is already present before the timeout boundary.

The run then attempts the next normalization step: polar decomposition $\mathbb{C}^\times \simeq \mathbb{R}_{>0} \times U(1)$, construction of the unit-modulus central subgroup, splitting of the positive modulus from the full carrier, the normalized quotient $(U(1) \times \mathbb{H}_1) / \Delta \mathbb{Z}_2$, and finally a direct equivalence with $U(2)$. Exactly this file contains all twenty-four unresolved proof placeholders. Three later files—`SignStructures.lean`, `TorsorPlacement.lean`, and `NegativeControls.lean`—contain no `sorry` themselves, but import the unfinished normalization module and therefore cannot yet be treated as verified Task-21 endpoints.

This creates an unusually precise verification boundary. The new source puts the project much closer to the classical spin environment: at source level it contains the group-theoretic

ingredients of the standard quaternionic double cover, namely a unit-quaternion/ $SU(2)$ core, an $SO(3)$ visible target, a quaternion-conjugation projection, and kernel $\{\pm 1\}$. In established mathematics the unit quaternions are $SU(2) \cong Spin(3)$ and quaternionic conjugation gives the standard double cover $Spin(3) \rightarrow SO(3)$ [3, 7, 14, 4]. But a *spin structure* is a further bundle-level lift, not merely the existence of the group $Spin(3)$ and its covering map. No frame bundle, principal Spin bundle, Stiefel–Whitney class, or bundle lifting problem has been constructed here. The correct statement is therefore that Task 21 has moved the reconstruction very close to the *group-theoretic substrate* of spin geometry, while the geometric notion of a spin structure remains unreached.

Because no Task-21 build was completed, none of these new source-level theorems is yet promoted to a certified project endpoint in this paper. Task 22 is deliberately restricted to completion and verification: eliminate or correct the existing placeholders in the normalization file, restore a complete import chain, create the final Task-21 endpoint and audit, run focused and whole-project builds, run the prohibited-construct scan and axiom audit, and make no new mathematical extension. The purpose of Task 22 is to determine whether the already-written Task-21 mathematics actually compiles as a coherent formal development.

Keywords: split-complex Lorentz structure; interrupted Lean run; unit quaternions; $SU(2)$; $SO(3)$; $Spin(3)$; quaternionic double cover; central product; $\mathbb{C}^\times$; $Spin^c(3)$ candidate; section obstruction; formal verification; Conservation of Difficulty.

Contents

1	Why Part XXI must be an interrupted-run paper	4
2	Archive provenance and exact verification boundary	4
3	The source dependency chain and where it stops	5
4	Task XX as the launch point	6
5	The quaternion core is repackaged as an exact group object	6
6	The source constructs an explicit $SU(2)$ model	7
7	The visible group is taken all the way to $SO(3)$ in source	7
8	The core projection is compared with quaternionic conjugation	8
9	The no-go becomes a section theorem	8
10	The exact centre is source-identified with $\mathbb{C}^\times$	9
11	The full carrier is already written as a central product	9
12	How close is this to spin geometry?	10
12.1	Level I: the spin group carrier	10
12.2	Level II: the standard double-cover projection	10
12.3	Level III: a spin structure	10
13	The exact timeout boundary: normalization of the central extension	10
13.1	Polar decomposition of the centre	10
13.2	The normalized central subgroup	11
13.3	Splitting off the positive modulus	11
13.4	The normalized quotient and $U(2)$	11

14 Later source files: useful, but transitively build-pending	11
14.1 The two different $\mathbb{Z}_2$ structures	11
14.2 The half-odd labels as phase weights	12
14.3 Negative controls	12
15 What Task XXI already changes conceptually	12
16 Formal status ledger	13
17 Task XXII: completion and verification only	13
17.1 1. Preserve the existing Task-21 source boundary	14
17.2 2. Finish <code>NormalizedCentralCarrier.lean</code>	14
17.3 3. Build the downstream source exactly as written	14
17.4 4. Create the missing Task-21 endpoint	14
17.5 5. Run focused and whole-project builds	15
17.6 6. Re-run the proof-escape audit	15
17.7 7. Produce the missing Task-21 audit	15
17.8 8. Stop	15
18 What would be established if Task XXII succeeds	16
19 Conclusion	16
A Principal Task-21 source declarations by module	17
A.1 Group packaging	17
A.2 Quaternion and $SU(2)$ identification	17
A.3 Visible $SO(3)$ identification and projection comparison	17
A.4 Section, centre, and central product	18
A.5 Post-timeout-dependent source declarations	18
B Exact distribution of the 24 unresolved proof placeholders	18
C Frozen Task-XXII file-level acceptance sequence	19

1 Why Part XXI must be an interrupted-run paper

Part XX changed the character of the reconstruction. The broad question “what does the accumulated object resemble?” was replaced by a finite set of explicit group-theoretic tests. The intrinsic core had already been identified exactly with the unit quaternions; the visible quotient had already been constructed; the core kernel was already $\{\pm 1\}$; and the larger carrier already had a continuous central kernel. Task 21 was therefore asked to close a short list of concrete diagrams rather than continue broad exploration [13, 1].

The run did not reach its normal completion protocol. That fact changes how its output must be reported. In previous parts, a green Aristotle archive could be separated into three layers:

1. a source-level mathematical claim;
2. a successful Lean build of that claim;
3. a final audit exposing principal endpoints and their axioms.

Task 21 provides only the first layer for its new mathematics. It supplies source text, but no final Task-21 build or endpoint audit.

Methodological principle 1 (Interrupted-run reporting rule). A source theorem from an interrupted run may be mathematically substantial and worth analyzing, but it must not be reported as a certified project theorem until the corresponding import chain compiles and the endpoint survives the final audit.

This paper therefore uses three distinct statuses:

SOURCE-CLOSED

the relevant source module contains a complete proof term and no `sorry`, but Task 21 did not produce a final build that certifies the module in the complete chain;

INCOMPLETE the source itself contains unresolved proof placeholders or an unfinished construction;

BUILD-PENDING

a source statement appears complete, but depends transitively on the unfinished module and therefore cannot yet be promoted to a verified endpoint.

That distinction is the central provenance fact of Part XXI.

2 Archive provenance and exact verification boundary

The supplied interrupted archive is recorded as [2]. Its SHA-256 is

52a12cb2e10e0afb1e5e2ee7ace1029d4e3b536dfc355ef6511115cf155ce2f7.

The inherited project remains pinned to

`leanprover/lean4:v4.28.0`

and mathlib revision

8f9d9cff6bd728b17a24e163c9402775d9e6a365.

The archive contains a directory `RequestProject/NullSectorTask21/` with thirteen Lean modules and 2655 lines in total. It contains no `Task21.lean`. There is no `TASK21_AUDIT.md`. The top-level README and Aristotle summary stop at Task 20. No Task-21 focused build result, whole-project build result, warning report, or `#print axioms` inventory was produced.

A static scan of the Task-21 directory gives the exact unresolved-proof boundary:

- twelve Task-21 modules contain no `sorry`;
- all unresolved proof placeholders occur in `NormalizedCentralCarrier.lean`;
- that file contains twenty-four occurrences of `sorry` on twenty-two source lines;
- no later source file contains its own `sorry`, but the later chain imports the unfinished module.

Proof boundary 1 (What was and was not verified for this paper). *The manuscript environment inspected the archive, module dependency chain, theorem declarations, source status, and placeholder locations. It did not independently run Lean 4.28.0 on the Task-21 code. Consequently, every new Task-21 theorem discussed below is explicitly source-level until Task 22 completes the build and audit.*

The last fully certified baseline is Task 20, whose supplied archive reported a successful focused build, a successful 8249-job whole-project build, no prohibited proof escapes, and nineteen principal endpoints with axioms exactly `[propext, Classical.choice, Quot.sound]` [1, 13]. Task 21 starts from that green baseline but does not itself return to green.

3 The source dependency chain and where it stops

The Task-21 import chain is almost linear:

$$\begin{aligned}
&\text{SafeBase} \rightarrow \text{GroupPackaging} \rightarrow \text{CoreQuaternionEquiv} \rightarrow \text{SU2Model} \\
&\quad \rightarrow \text{VisibleRotationCandidate} \rightarrow \text{CoreProjection} \rightarrow \text{SectionObstruction} \\
&\quad \rightarrow \text{CentralComplex} \rightarrow \text{CentralProduct} \rightarrow \boxed{\text{NormalizedCentralCarrier}} \\
&\quad \rightarrow \text{SignStructures} \rightarrow \text{TorsorPlacement} \rightarrow \text{NegativeControls}.
\end{aligned} \tag{1}$$

The boxed module is the only source file containing unresolved proofs. This makes the interruption mathematically informative: the run did not fail while attempting to recognize the quaternion core, construct $SO(3)$, or compute the full central-product kernel. It reached all of those source statements first. The timeout occurred when trying to split the continuous central factor into modulus and phase and identify the normalized quotient with $U(2)$.

Module	Lines	Source status	Main role
SafeBase	112	SOURCE-CLOSED	coordinate and spatial-carrier lemmas
GroupPackaging	321	SOURCE-CLOSED	groups <code>LiftG</code> , <code>CUnitG</code> , <code>LiftZG</code> , projections and kernels
CoreQuaternionEquiv	218	SOURCE-CLOSED	unit quaternions $\leftrightarrow \mathcal{L}$, group equivalence and homeomorphism
SU2Model	162	SOURCE-CLOSED	explicit quaternion matrix model and $\mathcal{L} \simeq SU(2)$
VisibleRotationCandidate	549	SOURCE-CLOSED	spatial action, $SO(3)$ membership, surjectivity, $G_{\text{vis}} \simeq SO(3)$
CoreProjection	130	SOURCE-CLOSED	quaternion conjugation map, commuting projection statement, kernel $\{\pm 1\}$
SectionObstruction	138	SOURCE-CLOSED	set-theoretic section, continuous-section no-go
CentralComplex	189	SOURCE-CLOSED	$\mathcal{C}^\times \simeq \mathbb{C}^\times$
CentralProduct	255	SOURCE-CLOSED	centre/core intersection, product kernel, quotient description of $\mathcal{L}_Z$

Module	Lines	Source status	Main role
NormalizedCentralCarrier	135	INCOMPLETE	polar split, normalized subgroup, $U(2)$ comparison; 24 placeholders
SignStructures	147	BUILD-PENDING	compares core sign with direction reversal
TorsorPlacement	154	BUILD-PENDING	locates half-odd labels as central phase weights
NegativeControls	145	BUILD-PENDING	anti-overclaim counterexamples

The status “source-closed” deliberately does not mean “Lean-verified in Task 21.” It means only that the file itself contains no explicit proof hole and was written before the unique unfinished module in the import chain.

4 Task XX as the launch point

Task 20 had already contracted the translation problem to three explicit candidate identifications:

1. identify the quaternionic core with a standard named compact group;
2. identify the visible image and prove that the intrinsic projection is the standard quaternionic rotation map;
3. identify the enlarged continuously central carrier as a precise central product, and only then test a normalized $Spin^c(3)/U(2)$ comparison.

The important methodological point was that these candidates were not to be assumed. Unit quaternions had been identified formally in Task 20, but the visible $SO(3)$ target and the full central-product quotient were still comparison targets rather than project theorems [13].

Task 21’s interrupted source is therefore significant because it addresses exactly these three obligations, in order, and reaches source-level closure of the first two plus most of the third before the timeout.

5 The quaternion core is repackaged as an exact group object

The module `CoreQuaternionEquiv.lean` starts from the Task-20 map from Hamilton’s real quaternions into the reconstructed carrier. It defines the unit-quaternion subtype

$$\mathbb{H}_1 = \{q \in \mathbb{H} : \|q\| = 1\},$$

constructs the inverse coordinate map back from the intrinsic carrier, and packages the result as

$$\text{liftQuatEquiv} : \mathbb{H}_1 \simeq_{\text{Grp}} \mathcal{L}. \quad (2)$$

The same source also defines a homeomorphism between the unit-quaternion sphere and the underlying intrinsic Lift set.

This is an important strengthening of Part XX’s recognition theorem. Part XX showed that the intrinsic carrier is exactly the image of the norm-one quaternions. Task 21 writes the actual multiplicative equivalence and inverse in the group language needed by the subsequent quotient calculations.

Status 1 (Source-level core package). `liftQuatEquiv`, its inverse identities, multiplication/inverse compatibility, and `liftQuatHomeo` are present with no `sorry` in their source module. Their Task-21 build status remains pending.

6 The source constructs an explicit $SU(2)$ model

The next module does not merely attach the conventional name $SU(2)$ to the quaternion sphere. It constructs the standard 2×2 complex matrix of a quaternion, proves its multiplicative law and determinant formula, establishes unitarity and determinant one on norm-one quaternions, and packages both injectivity and surjectivity onto the mathlib special unitary group.

The principal source declarations are

$$\text{quatSUEquiv} : \mathbb{H}_1 \simeq_{\text{Grp}} SU(2), \quad (3)$$

and, by composition with equation (2),

$$\text{liftSUEquiv} : \mathcal{L} \simeq_{\text{Grp}} SU(2). \quad (4)$$

This matters for the spin comparison. Established mathematics identifies the unit quaternions, $SU(2)$, and $Spin(3)$ as isomorphic Lie groups [3, 7, 14]. The Task-21 source therefore does not merely look spinorial at this stage: if the module compiles, the project’s intrinsic core is formally equivalent to the conventional $SU(2)$ model, and hence classically to the group $Spin(3)$.

Methodological caveat 1 (Group identification is not yet a spin structure). *The statement $\mathcal{L} \cong SU(2) \cong Spin(3)$ identifies a group. A spin structure is additional geometric data: a lift of an oriented orthonormal frame bundle through $Spin(n) \rightarrow SO(n)$, subject to bundle compatibility. No such principal-bundle object is constructed in Task 21. The project is much closer to the group-theoretic substrate of spin geometry, but it has not yet crossed the bundle-level definition [8, 9].*

7 The visible group is taken all the way to $SO(3)$ in source

The largest new module, `VisibleRotationCandidate.lean`, contains 549 lines. It constructs the action of a visible transformation on the reconstructed three-dimensional spatial carrier, expresses that action as a 3×3 real matrix, and then proves the properties required to land in the special orthogonal group.

At source level the chain is explicit:

1. representative independence of the spatial action;
2. preservation of the reconstructed positive-definite spatial form;
3. preservation of the corresponding inner product;
4. compatibility with composition;
5. orthogonality of the matrix representative;
6. determinant +1;
7. injectivity of the map into the standard special orthogonal group;
8. surjectivity onto that group.

The final declaration is

$$\text{visibleRotationEquiv} : G_{\text{vis}} \simeq_{\text{Grp}} SO(3), \quad (5)$$

where the right-hand side is mathlib’s `Matrix.specialOrthogonalGroup (Fin 3) ℝ`.

If this source compiles, one of the largest residual obligations from Part XX is closed: the visible group is not merely rotation-like or a subgroup candidate; it is explicitly group-equivalent to the full standard $SO(3)$ model.

Proof boundary 2 (Current status of equation (5)). *The theorem is written with a complete proof term and no **sorry** in the interrupted archive. There is no Task-21 build report. Part XXI therefore records this as a source-level equivalence awaiting Task 22 certification.*

8 The core projection is compared with quaternionic conjugation

Once both ends of the prospective double cover are available, `CoreProjection.lean` compares the actual project map with the standard quaternion action. It defines the embedding of a spatial vector into the imaginary quaternion subspace, the quaternion-conjugation action

$$p \longmapsto qp\bar{q},$$

and the associated rotation matrix. It then states source-level equalities between this action and the intrinsic spatial action of the Task-20 projection.

The central declaration

$$\text{coreProjection_diagram} \tag{6}$$

packages the commutation of the intrinsic projection with the quaternionic rotation model. In the same module the source proves directly

$$\ker(\text{quaternion rotation}) = \{+1, -1\}. \tag{7}$$

This is precisely the check that Part XX demanded. A two-element kernel by itself was never enough to identify a covering map; Task 21 instead compares the action pointwise and then recovers the kernel in the standard model.

Classically, unit-quaternion conjugation on the imaginary quaternions is the standard surjective double-cover homomorphism

$$Spin(3) \cong SU(2) \longrightarrow SO(3), \quad \ker = \{\pm 1\}, \tag{8}$$

[7, 4, 14]. Therefore, if Packages A–E compile as written, the project has reached much more than “spin-like behavior”: it has source-level data matching the standard *group-level* $Spin(3) \rightarrow SO(3)$ double cover under explicit equivalences.

9 The no-go becomes a section theorem

Task 20 had already shown that quotient-compatible internal representatives exist set-theoretically but cannot be chosen continuously. The Task-21 source repackages this statement at the newly constructed group level.

The module `SectionObstruction.lean` contains:

- existence of a group-level set-theoretic section of the core projection;
- a definition of core sections on the visible image;
- a definition of quotient-compatible choices on the axis-parameter space;
- construction of a section from a quotient-compatible choice and conversely;
- equivalence of the corresponding continuity statements;
- `no_continuous_core_section`.

Thus the global obstruction is now expressed in the most classical-looking form reached by the project:

$$\exists \text{ set-theoretic section}, \quad \nexists \text{ global continuous section}. \tag{9}$$

If the projection diagram of section 8 builds, equation (9) becomes the project’s own internal route to the no-global-continuous-section property of the nontrivial quaternionic double cover. The earlier rate obstruction, relation-completeness obstruction, and representative-choice obstruction have therefore contracted to a conventional section problem at group level.

10 The exact centre is source-identified with $\mathbb{C}^\times$

The larger internal carrier cannot be reduced to the double-cover core because its kernel contains a continuous central plane. Task 21 next addresses that plane directly.

Writing a central element intrinsically as

$$z_c(a, b) = a \, 1 + bS, \quad S^2 = -1,$$

the source defines

$$\text{toComplex}(z_c(a, b)) = a + ib$$

and its inverse. Multiplicativity is proved from the inherited central multiplication law, coefficient uniqueness supplies injectivity, and invertibility is characterized by nonvanishing of the complex image.

The principal group equivalence is

$$\text{cUnitComplexEquiv} : \mathcal{C}^\times \simeq_{\text{Grp}} \mathbb{C}^\times. \quad (10)$$

This closes another near-identification from Part XX at source level. The continuous kernel is not merely complex-like: the written source gives an explicit multiplicative equivalence with nonzero complex numbers.

11 The full carrier is already written as a central product

The module `CentralProduct.lean` is the last source-closed module before the timeout boundary. It addresses the exact interaction between the discrete quaternionic core and the continuous complex centre.

First, it proves the intrinsic intersection statement

$$\mathcal{C}^\times \cap \mathcal{L} = \{+1, -1\}. \quad (11)$$

It then defines the multiplication homomorphism

$$\mu : \mathcal{C}^\times \times \mathcal{L} \longrightarrow \mathcal{L}_Z, \quad (z, g) \longmapsto zg,$$

states its surjectivity, and computes its kernel as the diagonal sign pair

$$\ker \mu = \{(1, 1), (-1, -1)\}. \quad (12)$$

The quotient is then packaged as

$$\text{centralProductEquiv} : (\mathcal{C}^\times \times \mathcal{L}) / \ker \mu \simeq_{\text{Grp}} \mathcal{L}_Z. \quad (13)$$

Finally, the source transports both factors to standard models, defining

$$\mu_{\text{std}} : \mathbb{C}^\times \times \mathbb{H}_1 \rightarrow \mathcal{L}_Z,$$

with the corresponding diagonal kernel, and gives

$$\text{liftZStandardEquiv} : (\mathbb{C}^\times \times \mathbb{H}_1) / \Delta \mathbb{Z}_2 \simeq_{\text{Grp}} \mathcal{L}_Z. \quad (14)$$

This is a substantive endpoint. It shows exactly why the full internal carrier cannot be described solely by the two-sheeted core cover. The project has, at source level, a classical compact quaternionic core sitting inside a larger central product with a noncompact continuous factor.

Methodological principle 2 (Two simultaneous structures). The reconstructed carrier can support a standard-looking $\mathbb{Z}_2$ double-cover core and a larger continuously central extension at the same time. Neither kernel should be used to erase the other.

12 How close is this to spin geometry?

At this point the phrase “closer to a spin structure” needs a precise hierarchy. There are three distinct levels.

12.1 Level I: the spin group carrier

Established mathematics identifies the unit quaternions with $SU(2)$ and $Spin(3)$ [3, 7, 8]. The Task-21 source contains an explicit $\mathcal{L} \simeq SU(2)$ equivalence. Subject to build verification, the intrinsic core is therefore at the conventional group $Spin(3)$ level.

12.2 Level II: the standard double-cover projection

The source also contains $G_{\text{vis}} \simeq SO(3)$ and a pointwise comparison of the intrinsic projection with quaternionic conjugation, together with the $\{\pm 1\}$ kernel. Subject to build verification, the project is therefore extremely close to—and source-level appears to contain—the standard group diagram

$$Spin(3) \longrightarrow SO(3).$$

This is much stronger than the $2\pi/4\pi$ analogy that motivated the earlier comparison.

12.3 Level III: a spin structure

A spin structure is not merely the group homomorphism in Level II. One needs an oriented orthonormal frame bundle over a base space and a compatible principal $Spin(n)$ -bundle lifting its structure group. The classical obstruction is expressed by the second Stiefel–Whitney class of that bundle [8, 9, 5].

No such bundle-level object is constructed in Task 21. There is no project theorem identifying a frame bundle, no principal $Spin(3)$ -bundle over a manifold, no bundle projection satisfying the spin-structure compatibility square, and no w_2 computation.

Status 2 (Exact spin-status statement). Task 21 is source-level very close to identifying the *group-theoretic double-cover substrate* used in three-dimensional spin geometry. It has not yet constructed a *spin structure*. The distance remaining is categorical, not merely terminological: one must pass from a group and its cover to a bundle-lifting problem over a base.

This is why the results are simultaneously more substantial than “spin-like” and still short of “spin structure derived.”

13 The exact timeout boundary: normalization of the central extension

The interruption occurs in `NormalizedCentralCarrier.lean`, a 135-line module that begins only after equation (14) has been written. The file contains twenty-four occurrences of `sorry`, all Task-21 proof holes in the archive.

The intended chain is transparent from the declarations already present in the file.

13.1 Polar decomposition of the centre

The first unfinished definition is

$$\text{polarEquiv} : \mathbb{C}^\times ? \simeq_{\text{Grp}} \mathbb{R}_{>0} \times U(1). \quad (15)$$

Its projections are intended to be the ordinary modulus and normalized phase.

13.2 The normalized central subgroup

The source then defines the set of unit-modulus central elements, attempts to package it as a subgroup, and defines the normalized full carrier consisting of products of unit-modulus central elements with the quaternionic core. The subgroup closure proofs remain placeholders.

13.3 Splitting off the positive modulus

The next intended principal equivalence is

$$\text{liftZSplit} : \mathcal{L}_Z \simeq_{\text{Grp}} \mathbb{R}_{>0} \times \mathcal{L}_{Z,1}. \quad (16)$$

This is the exact formal version of the heuristic decomposition suggested at the end of Part XX.

13.4 The normalized quotient and $U(2)$

The file then introduces a multiplication map

$$U(1) \times \mathbb{H}_1 \rightarrow \mathcal{L}_{Z,1},$$

intends to prove surjectivity and the diagonal-sign kernel, and thereby obtains a quotient expression for the normalized carrier. Finally, it defines the standard matrix map into $U(2)$ and intends to prove

$$\mathcal{L}_{Z,1} \simeq_{\text{Grp}} U(2). \quad (17)$$

Classically,

$$Spin^c(3) = (Spin(3) \times U(1))/\mathbb{Z}_2 \cong U(2)$$

[6, 10, 12, 11]. Thus the unfinished file is exactly where the broader full carrier would have been normalized into the standard $Spin^c(3)$ environment. But it did not finish, and the project must not claim this identification before Task 22 actually closes and builds it.

Proof boundary 3 (Timeout location). *The timeout did not occur in the $SU(2)$ construction, the $SO(3)$ construction, the quaternionic projection comparison, the section theorem, the complex-centre identification, or the full central-product kernel calculation. It occurred at the subsequent modulus/phase normalization and $U(2)$ comparison.*

14 Later source files: useful, but transitively build-pending

Three files occur after the unfinished normalization module. They contain no `sorry` themselves. Nevertheless, their import chain passes through `NormalizedCentralCarrier.lean`; they therefore cannot be counted as independently certified even at the module-chain level.

14.1 The two different $\mathbb{Z}_2$ structures

`SignStructures.lean` distinguishes:

1. the core sign $g \mapsto -g$, invisible under the projection and corresponding to a 2π parameter shift;
2. direction reversal $n \mapsto -n$, coupled to parameter reversal rather than to the same internal sign action.

The source explicitly states that the two structures are distinct and that the direction-reversal quotient has at least three elements, so it cannot simply be the two-element core kernel.

This is an important anti-overclaim result. Even if the core double cover becomes the classical $Spin(3) \rightarrow SO(3)$ cover, the antipodal direction quotient is not thereby identified with its kernel.

14.2 The half-odd labels as phase weights

`TorsorPlacement.lean` places the Task-20 torsor inside the now identified complex centre. At source level it states

$$\text{toComplex}(\text{labelBridge}(b, \theta)) = \exp(ib\theta). \quad (18)$$

Thus the label b is interpreted as the weight of a one-parameter circle-valued central character. The source then proves the full-turn equivalence

$$e^{i2\pi b} = e^{i2\pi b'} \iff b - b' \in \mathbb{Z}, \quad (19)$$

and restates the half-odd label difference theorem in exactly this language.

The source also makes the Task-20 information-loss theorem concrete: all half-odd labels produce the same central full-turn value -1 , while different half-odd weights remain distinguishable at other parameter values. The $\mathbb{Z}_2$ sign remembers only a quotient of the full weight data.

14.3 Negative controls

`NegativeControls.lean` preserves the intended anti-overclaim discipline. Among its source statements are noninjectivity of the core projection, explicit noncommuting visible transformations, and a counterexample showing that knowledge of a two-element kernel alone does not determine a homomorphism.

These later modules are mathematically coherent with the preceding source story, but Task 22 must build them after repairing the normalization import before any of them becomes a registered project result.

15 What Task XXI already changes conceptually

Even without a final build, the source has changed the geometry of the open problem. Part XIX had an explanatory excess: many familiar mathematical structures appeared simultaneously, and it was unclear whether they represented one object or several coupled ones. Part XX found the unit-quaternion anchor and reduced the problem to explicit isomorphism tests.

The interrupted Task-21 source goes further:

$$\begin{array}{llll} \text{intrinsic core} & \rightsquigarrow & \mathbb{H}_1 \simeq SU(2), \\ \text{visible group} & \rightsquigarrow & SO(3), \\ \text{intrinsic projection} & \rightsquigarrow & \text{quaternion conjugation}, \\ \text{core kernel} & \rightsquigarrow & \{\pm 1\}, \\ \text{continuous centre} & \rightsquigarrow & \mathbb{C}^\times, \\ \text{full carrier} & \rightsquigarrow & (\mathbb{C}^\times \times \mathbb{H}_1)/\Delta\mathbb{Z}_2. \end{array} \quad (20)$$

If Task 22 certifies these declarations, the principal broad identification problem will be almost entirely closed at group level. Conservation of Difficulty then moves to a much sharper categorical boundary:

Why did the intrinsic reconstruction produce the standard group-theoretic double cover used by spin geometry, and what additional base-space/bundle data would be required before this becomes a spin structure rather than merely its structure-group substrate?

This is a qualitatively different question from the earlier search for a spin-like resemblance. It is no longer asking whether the algebra resembles spin. It is asking how far a formally identified $Spin(3) \rightarrow SO(3)$ -type group diagram can and should be lifted into geometry.

Methodological principle 3 (Conservation of Difficulty after the interrupted run). The current difficulty is not another algebraic discovery task. It is verification first, and only after verification a categorical decision about whether the next object to reconstruct is a bundle lift or whether the group-level result is the natural endpoint of the present kinematic problem.

16 Formal status ledger

The following table intentionally separates source content from certification.

Claim or object	Part-XXI status
Task-21 final endpoint module	ABSENT
Task-21 audit	ABSENT
Task-21 focused build	NOT REPORTED
Task-21 whole-project build	NOT REPORTED
Task-21 axiom audit	NOT REPORTED
LiftG group packaging	SOURCE-CLOSED
$\mathcal{L} \simeq$ unit quaternions as multiplicative equivalence	SOURCE-CLOSED
Topological equivalence of core with unit-quaternion sphere	SOURCE-CLOSED
$\mathcal{L} \simeq SU(2)$	SOURCE-CLOSED
$G_{\text{vis}} \simeq SO(3)$	SOURCE-CLOSED
Intrinsic spatial action = quaternion conjugation action	SOURCE-CLOSED
Projection comparison diagram	SOURCE-CLOSED
Quaternionic rotation kernel = $\{\pm 1\}$	SOURCE-CLOSED
Set-theoretic section exists	SOURCE-CLOSED
No continuous core section	SOURCE-CLOSED
$\mathcal{C}^\times \simeq \mathbb{C}^\times$	SOURCE-CLOSED
$\mathcal{C}^\times \cap \mathcal{L} = \{\pm 1\}$	SOURCE-CLOSED
$\mathcal{L}_Z \simeq (\mathcal{C}^\times \times \mathcal{L})/\Delta\mathbb{Z}_2$	SOURCE-CLOSED
$\mathcal{L}_Z \simeq (\mathbb{C}^\times \times \mathbb{H}_1)/\Delta\mathbb{Z}_2$	SOURCE-CLOSED
$\mathbb{C}^\times \simeq \mathbb{R}_{>0} \times U(1)$ inside Task 21	INCOMPLETE
Normalized central subgroup/group	INCOMPLETE
$\mathcal{L}_Z \simeq \mathbb{R}_{>0} \times \mathcal{L}_{Z,1}$	INCOMPLETE
$\mathcal{L}_{Z,1} \simeq (U(1) \times \mathbb{H}_1)/\Delta\mathbb{Z}_2$	INCOMPLETE
$\mathcal{L}_{Z,1} \simeq U(2)$	INCOMPLETE
$Spin^c(3)$ project identification	NOT FORMALIZED
Core sign versus direction reversal distinction	BUILD-PENDING
Half-odd labels as $e^{ib\theta}$ phase weights	BUILD-PENDING
Full-turn equality iff integer weight difference	BUILD-PENDING
Task-21 negative controls	BUILD-PENDING
Principal spin structure on a bundle	NOT CONSTRUCTED
w_2 obstruction	NOT CONSTRUCTED
Physical spin degree of freedom	NOT DERIVED

17 Task XXII: completion and verification only

Task 22 must be deliberately narrower than any previous task in this sequence. It is not a successor mathematics task. It is a completion and certification pass over the interrupted Task-21 source.

Methodological principle 4 (Task-XXII freeze). Task 22 must not introduce a new mathematical branch, new physical interpretation, new representation target, new topology problem, or new bundle construction. Its job is to determine whether the Task-21 source already present in the archive can be completed and built coherently.

The required work is finite.

17.1 1. Preserve the existing Task-21 source boundary

Use the interrupted Task-21 archive as the input state. Do not redesign the already source-closed modules merely to obtain a shorter proof. Repairs are allowed only where needed to make the existing claims typecheck and compile.

17.2 2. Finish `NormalizedCentralCarrier.lean`

Eliminate all twenty-four existing `sorry` occurrences. The intended statements are already fixed by the file:

- polar decomposition of $\mathbb{C}^\times$;
- subgroup closure of the unit-modulus centre;
- subgroup closure of the normalized carrier;
- positive-modulus central homomorphism;
- splitting $\mathcal{L}_Z \simeq \mathbb{R}_{>0} \times \mathcal{L}_{Z,1}$;
- normalized multiplication map from $U(1) \times \mathbb{H}_1$;
- diagonal-sign kernel;
- normalized quotient equivalence;
- standard matrix map into $U(2)$;
- surjectivity and matching kernel;
- normalized $U(2)$ equivalence.

If an intended statement is false, the correct Task-22 action is to record the failure and replace it only by the strongest corrected statement necessary to make the Task-21 status truthful. It is not to open a new search.

17.3 3. Build the downstream source exactly as written

After the normalization module is complete, compile `SignStructures`, `TorsorPlacement`, and `NegativeControls`. Their current lack of local `sorry` is not sufficient; their transitive import chain must actually build.

17.4 4. Create the missing Task-21 endpoint

Add the principal Task-21 endpoint module exposing the intended final theorems from the completed source chain. This module should not contain new mathematics; it should only package the final declarations and run `#print axioms` on them.

17.5 5. Run focused and whole-project builds

Run, in order:

1. the focused Task-21 endpoint build;
2. the full project `lake build`.

Record exact job counts and all warnings. A failure at any stage is part of the Task-22 result and must be repaired before documentation is finalized.

17.6 6. Re-run the proof-escape audit

Scan the entire Task-21 source for at least

```
sorry, admit, axiom, implemented_by, native_decide, bv_decide, unsafe.
```

The completed Task-21 layer must contain no prohibited proof escape.

17.7 7. Produce the missing Task-21 audit

The audit must report:

- exact source order;
- theorem inventory;
- formal versus comparison-only modules;
- every repair made during Task 22;
- failed proof attempts that changed any theorem boundary;
- focused and full build results;
- warnings;
- axiom report;
- final decision table matching section 16.

17.8 8. Stop

Once Task 21 is green and audited, Task 22 is complete. It must not proceed to frame bundles, spin structures, characteristic classes, dynamics, or any new Task-23 mathematics.

Status 3 (Task-XXII acceptance criterion). The only desired new information from Task 22 is whether the already-written Task-21 claims survive completion and whole-project compilation. A green build upgrades source-level results to formal project results. A failed claim must be corrected and documented. No new mathematical scope is authorized.

18 What would be established if Task XXII succeeds

If Task 22 completes the file exactly along its intended lines and the entire project builds, the cumulative picture will become exceptionally sharp.

At group level the reconstruction would then contain:

$$\begin{aligned}
\mathcal{L} &\cong \mathbb{H}_1 \cong SU(2) \cong Spin(3), \\
G_{\text{vis}} &\cong SO(3), \\
\text{proj}|_{\mathcal{L}} &\cong \text{standard quaternionic double cover}, \\
\ker(\text{proj}|_{\mathcal{L}}) &= \{\pm 1\}, \\
\mathbb{C}^\times &\cong \mathbb{C}^\times, \\
\mathcal{L}_Z &\cong (\mathbb{C}^\times \times \mathbb{H}_1)/\Delta\mathbb{Z}_2.
\end{aligned} \tag{21}$$

If the interrupted normalization target also survives, one would further obtain

$$\mathcal{L}_Z \cong \mathbb{R}_{>0} \times U(2)$$

at the abstract group level, with the normalized factor matching the conventional quotient used for $Spin^c(3)$. The exact conventional naming should still follow the formal objects rather than precede them.

This would explain much of the earlier apparent strangeness. The $2\pi/4\pi$ behavior, central sign, quaternionic core, $SU(2)$ model, $SO(3)$ visible action, and no-continuous-section statement would no longer be separate analogies. They would be different manifestations of one explicit classical double-cover structure reconstructed from the earlier split-complex/Lorentz chain.

Yet even this strongest expected Task-22 outcome would still not prove that spacetime in the project carries a spin structure. It would prove that the kinematic implementation problem has reconstructed the relevant structure groups and covering homomorphism. Moving from those groups to a spin structure requires a new object: a suitable principal $SO(3)$ or $SO(1,3)$ frame-type bundle over a base together with a compatible lift. Whether the reconstruction itself supplies such a bundle is a later question and is deliberately outside Task 22.

19 Conclusion

Task 21 timed out, but it did not time out early. The interrupted archive contains a large, coherent-looking mathematical development and a sharply localized unfinished file. Before the interruption, the source had already repackaged the unit-quaternion core as a multiplicative equivalence, constructed an explicit $SU(2)$ model, constructed and surjected onto the full standard $SO(3)$ visible group, compared the intrinsic projection pointwise with quaternionic conjugation, recovered the $\{\pm 1\}$ kernel, turned the representative no-go into a continuous-section obstruction, identified the continuous centre with $\mathbb{C}^\times$, and written the full carrier as a central product with diagonal sign kernel.

The timeout occurs only when the source tries to normalize that larger central product: split $\mathbb{C}^\times$ into modulus and phase, separate the positive scalar factor, identify the unit-modulus quotient, and map it to $U(2)$. All twenty-four unresolved proof placeholders are confined to that one module. Later sign/torsor/control modules contain no local proof holes but remain transitively dependent on it.

This is a substantive Task-21 result even before certification because it pinpoints exactly what the agent had managed to construct. It also changes the status of the spin comparison. The project is no longer merely collecting spinorial symptoms. The source now contains the ingredients of the standard three-dimensional quaternionic double-cover diagram itself. That is much closer to spin geometry than the earlier $2\pi/4\pi$ analogy. But the distinction remains decisive:

reconstructing $Spin(3)$, $SO(3)$ and their covering map is a group-level result; constructing a spin structure requires an additional bundle-level lift that has not yet appeared.

The correct next move is therefore unusually conservative. Task 22 must not continue the mathematical exploration. It must finish the interrupted normalization module, compile the downstream files, create the missing Task-21 endpoint and audit, run focused and full builds, and report the axiom boundary. Only after that green build will the project know whether the substantial Task-21 source actually constitutes a valid formal result.

In Conservation-of-Difficulty terms, the difficulty has moved from discovery to certification. The mathematics has become specific enough that another broad search would be premature. The immediate scientific obligation is simply to find out whether the code already written is true in the strongest sense available to this project: whether Lean accepts the entire dependency chain.

A Principal Task-21 source declarations by module

This appendix records the principal declarations actually present in the interrupted archive. It is intentionally a source inventory rather than a proof audit. The declarations below are useful because Task 22 can use them as a fixed acceptance list instead of reconstructing the scope from prose.

A.1 Group packaging

Declaration	Source role
<code>LiftG</code>	intrinsic core packaged as a subgroup of invertible internal elements
<code>CUnitG</code>	invertible exact centre packaged as a subgroup
<code>LiftZG</code>	full central carrier packaged as a subgroup
<code>GvisG</code>	visible image packaged as a subgroup of invertible endomorphisms
<code>projCore</code>	core projection $\mathcal{L} \rightarrow G_{\text{vis}}$
<code>projZ</code>	full-carrier projection $\mathcal{L}_Z \rightarrow G_{\text{vis}}$
<code>ker_projCore</code>	intrinsic source theorem giving the two-element core kernel
<code>ker_projZ</code>	intrinsic source theorem giving the full central kernel
<code>projCore_surjective</code>	source-level surjectivity of the core projection
<code>projZ_surjective</code>	source-level surjectivity of the full projection

A.2 Quaternion and $SU(2)$ identification

Declaration	Source role
<code>liftQuatEquiv</code>	multiplicative equivalence $\mathbb{H}_1 \simeq \mathcal{L}$
<code>liftQuatHomeo</code>	homeomorphism of the unit-quaternion sphere with the intrinsic Lift set
<code>quatMat_mul</code>	multiplicativity of the standard 2×2 complex quaternion matrix
<code>quatMat_det</code>	determinant equals quaternion norm square
<code>quatMat_mem_SU</code>	norm-one quaternions land in the standard special unitary group
<code>toSU_injective</code>	injectivity into $SU(2)$
<code>toSU_surjective</code>	surjectivity onto $SU(2)$
<code>quatSUEquiv</code>	multiplicative equivalence $\mathbb{H}_1 \simeq SU(2)$
<code>liftSUEquiv</code>	multiplicative equivalence $\mathcal{L} \simeq SU(2)$

A.3 Visible $SO(3)$ identification and projection comparison

Declaration	Source role
<code>spatAct_indep_of_representative</code>	spatial action independent of chosen internal lift
<code>spatAct_preserves_form</code>	preservation of the reconstructed spatial quadratic form
<code>spatAct_preserves_inner</code>	preservation of the spatial inner product
<code>rotMat_mem_orthogonalGroup</code>	visible matrix is orthogonal
<code>rotMat_det</code>	visible matrix has determinant +1
<code>rotMat_mem_SO3</code>	visible matrix lies in the standard special orthogonal group
<code>toRot_injective</code>	injectivity of the visible-to- $SO(3)$ homomorphism
<code>toRot_surjective</code>	surjectivity onto standard $SO(3)$
<code>visibleRotationEquiv</code>	group equivalence $G_{\text{vis}} \simeq SO(3)$
<code>spatAct_proj_fromQuat</code>	intrinsic projection action equals quaternion conjugation on vectors
<code>rotMat_proj_fromQuat</code>	equality of intrinsic and quaternionic rotation matrices
<code>coreProjection_diagram</code>	source-level commuting projection statement
<code>quatRot_kernel</code>	standard quaternionic rotation kernel is $\{\pm 1\}$

A.4 Section, centre, and central product

Declaration	Source role
<code>exists_group_section</code>	set-theoretic section of the core group projection
<code>exists_core_section</code>	set-theoretic section on the visible image
<code>continuous_section_iff_continuous_choice</code>	continuity translation between sections and quotient-compatible choices
<code>no_continuous_core_section</code>	no global continuous section
<code>cUnitComplexEquiv</code>	multiplicative equivalence $\mathcal{C}^\times \simeq \mathbb{C}^\times$
<code>CUnit_inter_Lift</code>	pointwise intersection theorem: only ± 1
<code>inter_CUnitG_LiftG</code>	packaged subgroup-intersection statement
<code>mu_surjective</code>	surjectivity of centre-times-core multiplication
<code>ker_mu</code>	diagonal two-element kernel
<code>centralProductEquiv</code>	quotient equivalence $(\mathcal{C}^\times \times \mathcal{L}) / \ker \mu \simeq \mathcal{L}_Z$
<code>ker_muStd</code>	diagonal sign kernel after transport to $\mathbb{C}^\times \times \mathbb{H}_1$
<code>liftZStandardEquiv</code>	quotient equivalence $(\mathbb{C}^\times \times \mathbb{H}_1) / \Delta \mathbb{Z}_2 \simeq \mathcal{L}_Z$

A.5 Post-timeout-dependent source declarations

The following declarations occur in files with no local `sorry`, but those files import `NormalizedCentralCarrier.lean` and hence remain build-pending:

Declaration	Source role
<code>projCore_sign</code>	core sign is invisible in the visible projection
<code>Un_neg_axis_neg_param</code>	simultaneous axis and parameter reversal leaves the reference implementer unchanged
<code>Un_add_two_pi</code>	2π parameter shift produces the negative core element
<code>two_sign_structures_distinct</code>	core sign and direction reversal are source-level distinct
<code>revQuot_not_two_element</code>	direction-reversal quotient is not the two-element core kernel
<code>toComplex_labelBridge</code>	central rate factor becomes $\exp(ib\theta)$
<code>full_turn_eq_iff_int_difference</code>	full-turn equality iff the weight difference is integral
<code>full_turn_remembers_only_parity</code>	full turn forgets the integer offset of half-odd labels
<code>kernel_does_not_determine_map</code>	negative control against kernel-only identification

B Exact distribution of the 24 unresolved proof placeholders

All Task-21 proof holes are confined to `NormalizedCentralCarrier.lean`. They are distributed as follows; the count is by occurrence, not by source line.

Declaration or construction	sorry count	Intended obligation
<code>polarEquiv</code>	1	polar group equivalence $\mathbb{C}^\times \simeq \mathbb{R}_{>0} \times U(1)$
<code>polarEquiv_fst</code>	1	modulus projection formula
<code>polarEquiv_snd</code>	1	phase projection formula
<code>CUnit1_subset_CUnit</code>	1	normalized centre lies in invertible centre
<code>CUnit1G</code>	3	identity, multiplication, inverse closure
<code>LiftZ1G</code>	3	identity, multiplication, inverse closure
<code>posCentral</code>	5	inverse witnesses and homomorphism laws
<code>liftZSplit</code>	1	positive-modulus direct-product split
<code>muNorm</code>	1	normalized multiplication homomorphism
<code>muNorm_surjective</code>	1	surjectivity onto normalized carrier
<code>ker_muNorm</code>	1	diagonal sign kernel
<code>nuU2</code>	3	unitary membership and homomorphism laws
<code>nuU2_surjective</code>	1	surjectivity onto $U(2)$
<code>ker_nuU2</code>	1	equality of normalized and $U(2)$ kernels
Total	24	all in one 135-line module

Two declarations in that module, `normalizedQuotientEquiv` and `normalizedU2Equiv`, contain no local `sorry` because they are assembled from preceding constructions. They remain incomplete because their dependencies contain the placeholders listed above.

C Frozen Task-XXII file-level acceptance sequence

For Task 22 the most conservative build sequence is the existing dependency order. No later module should be debugged before its predecessor is green:

1. build or re-check `SafeBase`;
2. `GroupPackaging`;
3. `CoreQuaternionEquiv`;
4. `SU2Model`;
5. `VisibleRotationCandidate`;
6. `CoreProjection`;
7. `SectionObstruction`;
8. `CentralComplex`;
9. `CentralProduct`;
10. finish and build `NormalizedCentralCarrier`;
11. build `SignStructures`;
12. build `TorsorPlacement`;
13. build `NegativeControls`;
14. create and build the missing `Task21.lean`;
15. run the whole-project build;

16. run the final prohibited-construct and axiom audits;
17. write `TASK21_AUDIT.md` only after the above are green.

Task 22 should preserve every failure encountered in this sequence as engineering provenance. If a source-closed Task-21 declaration turns out not to typecheck once reached, that is mathematically relevant and must be documented rather than hidden by rewriting the history.

References

- [1] Aristotle (Harmonic). *Task 20: Intrinsic Equivalence and Structure Audit*. Run 2bc3fc02-9b8a-4587-a8bc-41cbd377b212; Lean archive supplied by the editor; archive SHA-256 3f8712040e59d8b3d2f4b28af7a8b29bb1e1293b64027b2a6a1182e0e81f5ceb. 2026.
- [2] Aristotle (Harmonic). *Task 21: Core and Central-Extension Identification – Interrupted Run Archive*. Interrupted/time-limited Lean source archive supplied by the editor on 4 September 2026; no Task21 endpoint module, final audit, focused build report, full-project build report, or Task21 axiom report; archive SHA-256 52a12cb2e10e0afb1e5e2ee7ace1029d4e3b536dfc355ef6511115cf155ce2f7. 2026.
- [3] John H. Conway and Derek A. Smith. *On Quaternions and Octonions: Their Geometry, Arithmetic, and Symmetry*. A K Peters, 2003.
- [4] Encyclopedia of Mathematics. *Cayley–Klein parameters*. Describes the epimorphism and double covering $SU(2) \rightarrow SO(3)$; accessed 4 September 2026. URL: https://encyclopediaofmath.org/wiki/Cayley-Klein_parameters.
- [5] Encyclopedia of Mathematics. *Characteristic class*. Includes the spin lifting sequence and the second Stiefel–Whitney obstruction; accessed 4 September 2026. URL: https://encyclopediaofmath.org/wiki/Characteristic_class.
- [6] Robert E. Gompf and András I. Stipsicz. *4-Manifolds and Kirby Calculus*. Vol. 20. Graduate Studies in Mathematics. American Mathematical Society, 1999. DOI: 10.1090/gsm/020.
- [7] Brian C. Hall. *Lie Groups, Lie Algebras, and Representations: An Elementary Introduction*. 2nd ed. Vol. 222. Graduate Texts in Mathematics. Springer, 2015.
- [8] H. Blaine Lawson and Marie-Louise Michelsohn. *Spin Geometry*. Princeton University Press, 1989.
- [9] John W. Milnor and James D. Stasheff. *Characteristic Classes*. Princeton University Press, 1974.
- [10] Liviu I. Nicolaescu. *Notes on Seiberg–Witten Theory*. Vol. 28. Graduate Studies in Mathematics. American Mathematical Society, 2000.
- [11] nLab contributors. *spin^c*. Includes $Spin^c(3) \simeq U(2)$ and the quotient $(SU(2) \times U(1))/\mathbb{Z}_2$; accessed 4 September 2026. URL: <https://ncatlab.org/nlab/show/spin%E1%B6%9C>.
- [12] nLab contributors. *U(2)*. Records the exceptional isomorphism $Spin^c(3) \cong U(2)$ and its quotient description; accessed 4 September 2026. URL: <https://ncatlab.org/nlab/show/U%282%29>.
- [13] Jan Seeck, OpenAI ChatGPT, and Aristotle (Harmonic). *Split Complex Lorentzian Dynamics – Part XX: The Unit-Quaternion Core, the Exact Visible Quotient, and the Contraction of the Translation Gap*. Internal project manuscript, 4 September 2026. Sept. 2026.
- [14] Eric W. Weisstein. *Special Unitary Group*. Wolfram MathWorld. States the identification of $SU(2)$ with unit quaternions and the double cover $SU(2) \rightarrow SO(3)$; accessed 4 September 2026. URL: <https://mathworld.wolfram.com/SpecialUnitaryGroup.html>.